

Exercise 2: E-commerce Platform Search Function

1. Understand Big O Notation

Big O notation measures how the running time of an algorithm grows as input size increases.

Examples:

- $O(1)$ → Constant Time
- $O(\log n)$ → Logarithmic Time
- $O(n)$ → Linear Time
- $O(n^2)$ → Quadratic Time

2. Setup

```
class Product {  
    int productId;  
    String productName;  
    String category;  
  
    Product(int productId, String productName, String category) {  
        this.productId = productId;  
        this.productName = productName;  
        this.category = category;  
    }  
}
```

```
public String toString() {  
    return "ID: " + productId + " Name: " + productName + " Category: " + category;  
}  
}
```

3. Implementation

```
class Search {  
  
    public static void linearSearch(Product[] products, int targetId) {  
        for(int i = 0; i < products.length; i++) {  
            if(products[i].productId == targetId) {  
                System.out.println("Linear Search Result:");  
                System.out.println(products[i]);  
                return;  
            }  
        }  
    }  
    System.out.println("Product Not Found");  
}
```

```
}
```

```
public static void bubbleSort(Product[] products) {  
    int n = products.length;
```

```
    for(int i = 0; i < n - 1; i++) {  
        for(int j = 0; j < n - i - 1; j++) {
```

```
            if(products[j].productId > products[j + 1].productId) {
```

```
                Product temp = products[j];  
                products[j] = products[j + 1];  
                products[j + 1] = temp;
```

```
            }  
        }  
    }  
}
```

```
public static void binarySearch(Product[] products, int targetId) {  
    int left = 0;  
    int right = products.length - 1;
```

```
    while(left <= right) {
```

```
        int mid = left + (right - left) / 2;
```

```
        if(products[mid].productId == targetId) {  
            System.out.println("Binary Search Result:");  
            System.out.println(products[mid]);  
            return;  
        }
```

```
        else if(products[mid].productId < targetId) {  
            left = mid + 1;  
        }
```

```
        else {  
            right = mid - 1;  
        }  
    }
```

```
    System.out.println("Product Not Found");  
}
```

```
public class E_CommerceSearch {  
    public static void main(String[] args) {
```

```
Product[] products = {  
    new Product(104, "Phone", "Electronics"),  
    new Product(101, "Laptop", "Electronics"),  
    new Product(103, "Book", "Education"),  
    new Product(102, "Mouse", "Electronics")  
};
```

```
Search.linearSearch(products, 103);
```

```
Search.bubbleSort(products);
```

```
Search.binarySearch(products, 104);  
}  
}
```

4. Analysis

<i>Algorithm</i>	<i>Best</i>	<i>Worst</i>	<i>Average</i>
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Binary Search is more suitable because:

- It significantly reduces search time.
- Works efficiently for large product catalogs.
- Requires the data to be sorted beforehand.

Linear Search is suitable only for small datasets or unsorted data.